A machine-checked proof of the greedy Egyptian fraction theorem

Eric Spencer

November 2026

Abstract

The Fibonacci–Sylvester greedy algorithm writes a rational number $a/b \in (0,1)$ as a sum of distinct unit fractions by repeatedly subtracting the largest unit fraction that fits. We give a complete formalization in Lean 4 and Mathlib of the classical facts about it: the algorithm terminates after at most a steps, the unit fractions it produces sum to a/b, and their denominators strictly increase. As a corollary we obtain a formal proof that every rational in $(0,1)$ with numerator p in lowest terms is a sum of at most p distinct unit fractions. We also prove, on paper, that consecutive denominators satisfy $n_{i+1} \geq n_i(n_i-1)+1$, and we report the smallest fractions a/b that need the full a terms for $a \leq 14$.

1 Introduction

An *Egyptian fraction* is a sum of distinct unit fractions $1/n$. In the *Liber Abaci* (1202), Fibonacci described a method for writing any proper fraction this way: take the largest unit fraction not exceeding what is left, subtract it, and repeat. Sylvester [1] gave the first careful proof that the method stops, and the result now appears in most introductions to number theory.

The argument is short, but it combines three things that are easy to get subtly wrong: a ceiling, a truncated subtraction of natural numbers, and an induction on a quantity that is not the length of anything. That makes it a good test case for formalization. Our contribution is a complete, `sorry`-free Lean 4 proof [5] on top of Mathlib [4], organized so that each paper lemma corresponds to one Lean theorem.

2 The algorithm

Definition 1. For natural numbers a, b define a finite list $G(a,b)$ of natural numbers by $G(0,b) = [\,]$ and, for $a > 0$,

$$G(a,b) = n :: G(an-b,\ nb), \qquad n = \lceil b/a \rceil.$$

We work with the unreduced pair $(an-b,\ nb)$. Since $\lceil b/a \rceil = \lceil kb/ka \rceil$ for every $k \geq 1$, reducing the fraction at each step would produce exactly the same denominators, so nothing is lost. The definition is total: it makes sense for all a, b, and the recursive call always has a smaller first argument (Lemma 2). In Lean the definition is accepted by well-founded recursion on a, which is the formal statement that the algorithm terminates.

The minimality of $n = \lceil b/a \rceil$ is the inequality

$$(n-1)\,a < b \leq n\,a. \tag{1}$$

In Lean we avoid both the ceiling and the subtraction in (1) by writing the numerator as $a+1$ and using $n = \lfloor (b+a)/(a+1) \rfloor$, with the equivalent bounds $b \leq n(a+1) \leq b+a$ (`ceil_bounds`).

3 Main results

Lemma 2 (numerator descent)**.** *If $a > 0$ and $n = \lceil b/a \rceil$, then $an - b < a$.*

Proof. Immediate from the left inequality of (1). □

Theorem 3. *Let $a, b \in \mathbb{N}$ and $G(a, b) = [n_1, \ldots, n_k]$.*

1. *$k \le a$.* *(`greedy_length_le`)*
2. *If $b > 0$ then every $n_i \ge 1$ and $\sum_i 1/n_i = a/b$.* *(`greedy_pos`, `greedy_sum`)*
3. *Every n_i satisfies $b \le n_i\, a$.* *(`le_mul_of_mem_greedy`)*
4. *If $a < b$ then $n_1 < n_2 < \cdots < n_k$.* *(`greedy_strictly_increasing`)*

Proof. All four parts are proved by induction along the recursion, that is, by strong induction on a. Write $a' = an - b$ and $b' = nb$.

(1) Each step adds one term and, by Lemma 2, lowers a by at least one.

(2) If $b > 0$ then $n \ge 1$ by (1), so $b' > 0$ and the induction hypothesis applies. Since $na \ge b$, the subtraction $a' = na - b$ is exact, and $1/n + a'/b' = (b + na - b)/(nb) = a/b$.

(3) For the first term this is (1). A later term m lies in $G(a', b')$, so $nb \le ma'$ by induction. If $b = 0$ there is nothing to prove; otherwise $n \ge 1$ and $b \le nb \le ma' \le ma$.

(4) Let m be a term of $G(a', b')$. Then $a' > 0$, because $G(0, b')$ is empty. Part (3) gives $ma' \ge nb$, and $a' < a < b$ gives $nb > na'$, so $m > n$. The pair (a', b') again satisfies $a' < b'$, because $a' < b \le nb$, so induction finishes the proof. □

Corollary 4 (Fibonacci)**.** *Every rational $q \in (0, 1)$ is a sum of at most* $\mathrm{num}(q)$ *distinct unit fractions.* *(`exists_egyptian`)*

Proof. Write $q = p/d$ in lowest terms, so $0 < p < d$, and apply Theorem 3 to $G(p, d)$. □

4 Growth of the denominators

The greedy algorithm is famous for producing very large denominators. For example, the last denominator for $5/121$ has 25 digits, although $5/121 = 1/33 + 1/121 + 1/363$. The next proposition explains why.

Proposition 5. *If $0 < a < b$ and $G(a, b) = [n_1, \ldots, n_k]$, then $n_{i+1} \ge n_i(n_i - 1) + 1$ for $1 \le i < k$.*

Proof. Let r_{i-1} be the remainder before n_i is chosen, so $r_0 = a/b$ and $n_i = \lceil 1/r_{i-1} \rceil$. By Theorem 3(4), $n_i \ge n_1 \ge 2$. By minimality of n_i, $r_{i-1} < 1/(n_i - 1)$, so

$$r_i = r_{i-1} - \frac{1}{n_i} < \frac{1}{n_i - 1} - \frac{1}{n_i} = \frac{1}{n_i(n_i - 1)}.$$

Hence $n_{i+1} = \lceil 1/r_i \rceil > n_i(n_i - 1)$. □

Iterating, $n_{i+1} - \frac{1}{2} > (n_i - \frac{1}{2})^2$, so the denominators grow at least doubly exponentially, like Sylvester's sequence $2, 3, 7, 43, 1807, \ldots$. We checked the inequality by computer for every reduced fraction with denominator at most 400. It is not yet formalized.

5 The bound $k \le a$ is attained

The bound in Theorem 3(1) is sharp: for every a there are fractions a/b in lowest terms whose greedy expansion has exactly a terms [2]. Reaching a terms requires the numerator to drop by exactly one at every step. Since $a' = an - b \equiv -b \pmod{a}$, the first step drops it by one only when $b \equiv 1 \pmod{a}$, which makes the search below fast. Table 1 lists the smallest such b for $a \le 14$, together with the size of the last denominator.

a	least b	digits of n_a	a	least b	digits of n_a
2	3	1	9	271	396
3	7	3	10	1621	1160
4	17	7	11	199	1348
5	31	16	12	3961	5247
6	109	44	13	1769	8905
7	253	106	14	12013	24324
8	97	150			

Table 1: Smallest b with $\gcd(a, b) = 1$ such that $G(a, b)$ has exactly a terms. Computed by `scripts/stats.py` in [5].

6 The formalization

The development is a single file, `Egyptian/Basic.lean`, of under 200 lines. It builds against Mathlib with no `sorry`, and the main theorems depend only on Lean's standard axioms (`propext`, `Quot.sound`, and `Classical.choice` for the corollary). A continuous integration job rebuilds the proof on every change.

Three design choices kept the proofs short.

- *Pattern-match on $a + 1$.* Writing the numerator as $a + 1$ removes the side condition $a > 0$ and lets the ceiling be the plain division $\lfloor (b + a)/(a + 1) \rfloor$.
- *Natural numbers throughout the algorithm.* The list is computed on $\mathbb{N}$ and only the sum is cast to $\mathbb{Q}$. Truncated subtraction is harmless because `ceil_bounds` shows the subtraction is exact; `omega` then discharges every termination and inequality goal, treating products as atoms.
- *An invariant instead of a direct comparison.* Strict monotonicity is proved through Theorem 3(3), $b \le ma$, which survives the recursive call. Comparing neighbouring terms directly needs information about two steps at once and is harder to state as an induction.

The kernel also evaluates the definition: `greedy 4 13 = [4, 18, 468]` is checked by `decide`.

7 Further directions

- Formalize Proposition 5 and the doubly exponential lower bound.

- Formalize the sharpness result of Table 1: for every a there is a b with exactly a greedy terms.
- The *odd greedy algorithm* uses only odd denominators. Whether it terminates for every a/b with b odd is a well-known open problem [3]. Our definitions extend directly, and a formal search for a non-terminating case, or a proof of termination for small numerators, is a natural next step.
- The Erdős–Straus conjecture asks whether $4/n$ is always a sum of three unit fractions; the greedy algorithm alone does not settle it, but the same formal framework can check candidate identities.

References

[1] J. J. Sylvester, On a point in the theory of vulgar fractions, *Amer. J. Math.* **3** (1880), 332–335.

[2] M. Mays, A worst case of the Fibonacci–Sylvester expansion, *J. Combin. Math. Combin. Comput.* **1** (1987), 141–148.

[3] R. K. Guy, *Unsolved Problems in Number Theory*, 3rd ed., Springer, 2004, Section D11.

[4] The mathlib Community, The Lean mathematical library, *CPP 2020*, 367–381.

[5] E. Spencer, *greedy-egyptian-fractions*, `https://github.com/EricSpencer00/greedy-egyptian-fractions`, 2026.